

Heat Room Project

Krystian Jawny 275529, Emilia Szpyra 275518

June 15, 2026

1 Introduction

In order to simulate heat in a room, one imple way to do this is to calculate the temperature change step-by-step, second by second. While this is easy to code, it has a major flaw: if your virtual room has a lot of detail, a strict math rule forces you to use incredibly tiny time steps. If your time steps are even a fraction too large, the simulation breaks, and the temperatures spike to infinity. To fix this, we use a smarter approach called an "implicit method." This method uses a clever mathematical trick that keeps the simulation perfectly stable, allowing you to use much larger time steps without anything breaking. The catch is that this trick makes all the points in the room dependent on one another. Instead of calculating each point individually, you now have to solve a massive, interconnected puzzle of equations at every single tick of the clock. This report tests and compares four different computer methods used to solve that issue: Gaussian Elimination, LU Factorization, the Jacobi Method, and the Gauss-Seidel Method.

2 Mathematical Formulation & Discretization

2.1 The Governing PDE and Discretization

The transient distribution of temperature in a homogeneous two-dimensional domain without internal heat generation is governed by:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \quad (1)$$

where $T(x, y, t)$ is the temperature field and α is the thermal diffusivity coefficient.

Applying a first-order backward finite difference scheme to the time derivative, and second-order central differences to the spatial derivatives on a grid where $\Delta x = \Delta y = 1$, we get the implicit relation:

$$\frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t} = \alpha \left(T_{i+1,j}^{n+1} - 2T_{i,j}^{n+1} + T_{i-1,j}^{n+1} + T_{i,j+1}^{n+1} - 2T_{i,j}^{n+1} + T_{i,j-1}^{n+1} \right) \quad (2)$$

By defining the mesh Fourier parameter $\lambda = \alpha \Delta t / \Delta x^2$ and isolating the unknown terms at time step $n + 1$, we get the five-point computational stencil equation:

$$(1 + 4\lambda)T_{i,j}^{n+1} - \lambda T_{i-1,j}^{n+1} - \lambda T_{i+1,j}^{n+1} - \lambda T_{i,j-1}^{n+1} - \lambda T_{i,j+1}^{n+1} = T_{i,j}^n \quad (3)$$

2.2 System Assembly and Dirichlet Boundary Conditions

Mapping the 2D grid lexicographically into a 1D global state vector $\mathbf{T} \in \mathbb{R}^N$ (where $N = n_x \cdot n_y$ and index $k = i + (j - 1)n_x$) transforms the system into a sparse linear system solved sequentially at each time step:

$$\mathbf{A}\mathbf{T}^{n+1} = \mathbf{T}^n \quad (4)$$

The matrix $\mathbf{A}$ is a sparse pentadiagonal coefficient matrix with diagonal elements configured as:

$$A_{k,m} = \begin{cases} 1 + 4\lambda & \text{if } m = k \\ -\lambda & \text{if } m = k \pm 1 \text{ or } m = k \pm n_x \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

For Dirichlet boundary points, the boundary rows of $\mathbf{A}$ are modified so that off-diagonal entries are eliminated and the main diagonal entry equals 1 ($A_{k_{\text{bound}}, k_{\text{bound}}} = 1$). This ensures that the boundary nodes correctly satisfy the constant condition $T^{n+1} = T^n$.

3 Algorithmic Frameworks of the Solvers

3.1 Gaussian Elimination

Gaussian elimination operates via forward row reduction to achieve an upper triangular form, followed by back substitution. In MATLAB, this is implemented using the direct backslash operator ($\backslash$), which uses highly optimized sparse reordering techniques to minimize matrix fill-in during computation.

3.2 LU Factorization

LU Factorization decomposes the constant matrix $\mathbf{A}$ into lower ($\mathbf{L}$) and upper ($\mathbf{U}$) triangular matrices with row pivoting ($\mathbf{P}$): $\mathbf{PA} = \mathbf{LU}$. Because the coefficient matrix $\mathbf{A}$ does not change between time steps, this factorization is computed *only once* outside the main loop. Each time step is then reduced to efficient forward and backward substitutions:

$$\mathbf{L}\mathbf{y} = \mathbf{P}\mathbf{T}^n, \quad \mathbf{U}\mathbf{T}^{n+1} = \mathbf{y} \quad (6)$$

3.3 Jacobi Iterative Method

The Jacobi method splits $\mathbf{A}$ into a main diagonal matrix $\mathbf{D}$ and remaining off-diagonal elements $\mathbf{R}$. The element-wise update equation is:

$$T_i^{(m+1)} = \frac{1}{A_{i,i}} \left(T_i^n - \sum_{j \neq i} A_{i,j} T_j^{(m)} \right) \quad (7)$$

Because updates depend entirely on the values from the previous iteration step $\mathbf{T}^{(m)}$, this algorithm can be fully vectorized into native matrix-vector computations.

3.4 Gauss-Seidel Iterative Method

The Gauss-Seidel method updates values by immediately using newly calculated variables within the same spatial sweep:

$$T_i^{(m+1)} = \frac{1}{A_{i,i}} \left(T_i^n - \sum_{j=1}^{i-1} A_{i,j} T_j^{(m+1)} - \sum_{j=i+1}^N A_{i,j} T_j^{(m)} \right) \quad (8)$$

While this data dependency mathematically doubles the theoretical convergence rate compared to Jacobi, it introduces strict scalar dependencies that prevent vectorization.

4 Results and Visualizations

The model was evaluated using a 20×20 grid ($N = 400$), $\alpha = 0.5$, $\Delta t = 0.1$, $\lambda = 0.05$, a solver tolerance $\epsilon = 10^{-6}$, and an initial spatial heat spike of $T(10, 10) = 100^\circ\text{C}$, executed for 200 time steps.

4.1 Heat Distribution Visualizations

The final temperature profiles generated by the direct and iterative methods are documented below:

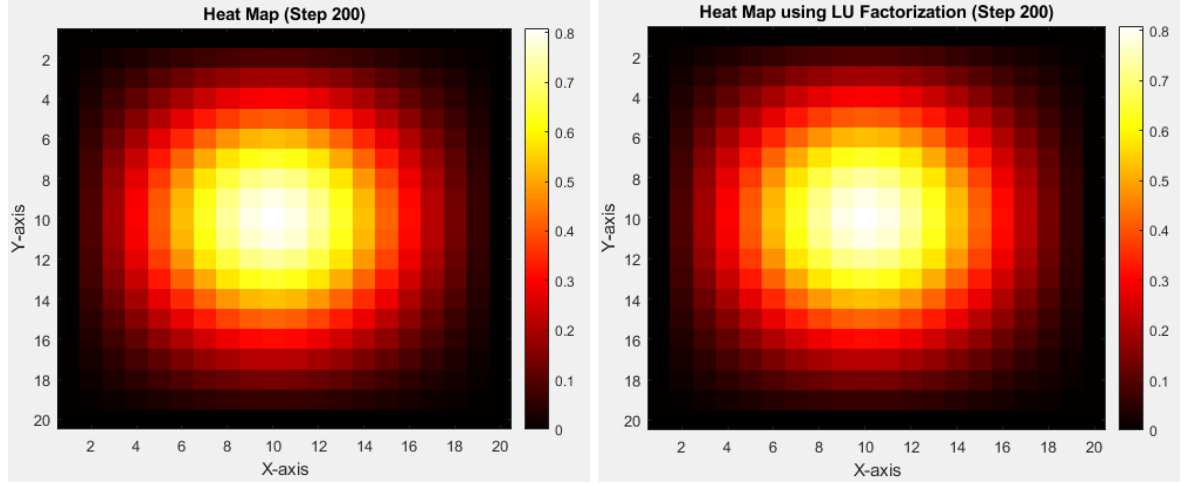


Figure 1: Temperature Heatmaps: Gaussian Elimination (left) and LU Decomposition (right) at Step 200.

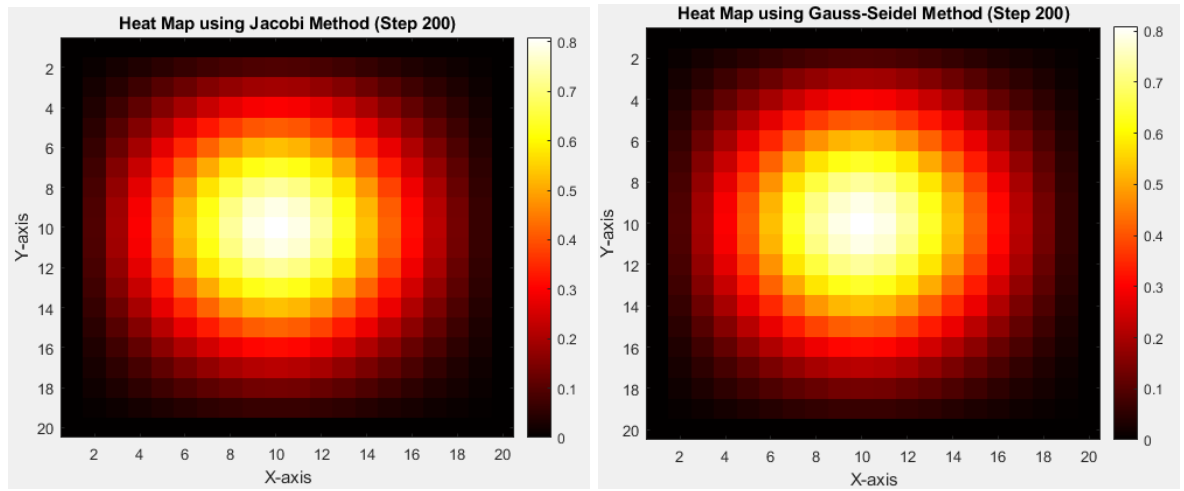


Figure 2: Temperature Heatmaps: Jacobi Method (left) and Gauss-Seidel Method (right) at Step 200.

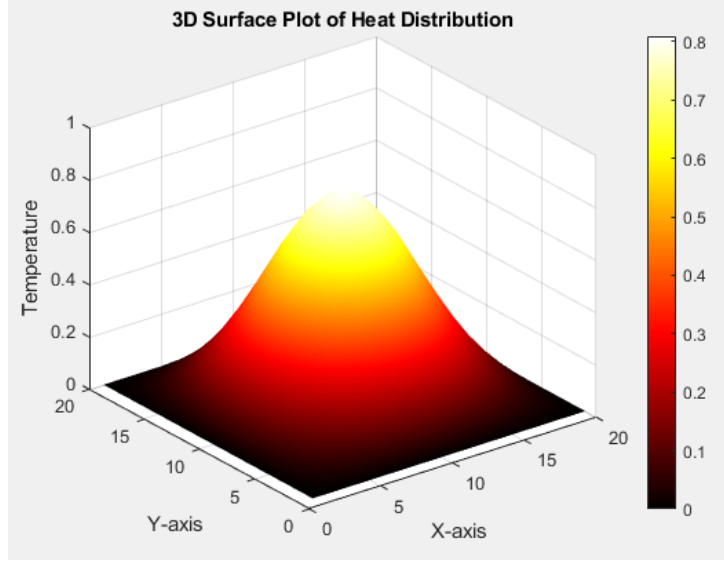


Figure 3: 3D Surface plot of the diffused hotspot temperature profile.

4.2 Solver Error and Deviation Maps

To verify accuracy, the absolute deviation between the direct baseline solver (Gaussian) and the iterative approximations was evaluated using difference mapping.

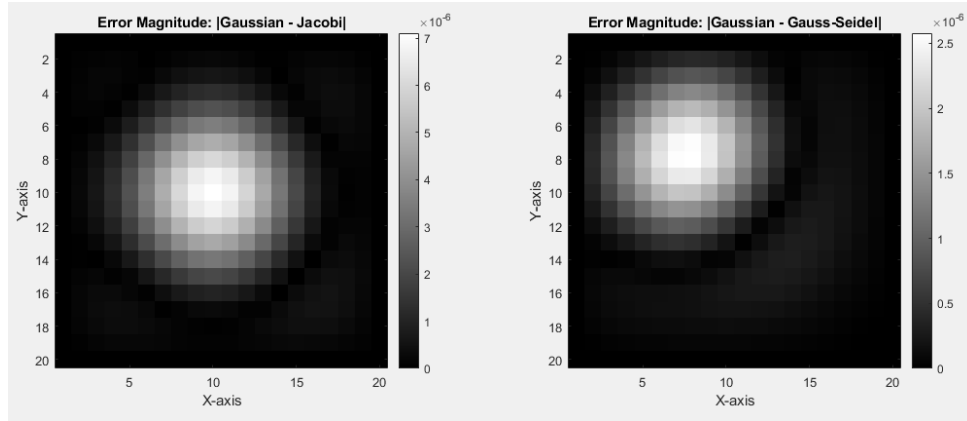


Figure 4: Absolute Error Distributions: $|Gaussian - Jacobi|$ (left) and $|Gaussian - Gauss-Seidel|$ (right).

When we look at where the calculation errors actually happen on the grid, we can see a clear difference in how these two methods work:

- The Jacobi Method distributes its error right in the middle, perfectly surrounding the original hot spot. Because it updates every point in the room at the exact same time using only old data, the heat spreads out evenly in all directions.
- The Gauss-Seidel Method shifts its error toward the top-left corner. This happens because of the way the computer reads the grid. Since the loop updates points one by one from left to right and top to bottom, the calculations constantly reuse brand-new data from the neighbors it just processed. This creates a directional bias, pulling the error toward the top-left starting corner.

5 Comparative Metrics & Discussion

The performance characteristics across all four methods are summarized in Tables 1 and 2.

Table 1: Computational Runtime and Iteration Performance

Solver Algorithm	Runtime (s)	Total Iterations
Gaussian Elimination ($\backslash$)	0.1604	-
LU Factorization (LU)	0.0039	-
Jacobi Method	0.0061	1337
Gauss-Seidel Method	3.4362	1113

Table 2: Absolute Error Bounds Measured Against Base Gaussian Solver

Comparison Configurations	Maximum Absolute Deviation (L_∞)
Gaussian vs. LU Factorization	0.0
Gaussian vs. Jacobi Method	7.1070×10^{-6}
Gaussian vs. Gauss-Seidel Method	2.5769×10^{-6}

The empirical performance data shows clear differences in execution efficiency:

- **LU Decomposition** proved to be the fastest strategy. Because the main matrix math is pre-calculated just once outside the time loop, each step only needs quick and simple triangular substitutions.
- **The Jacobi Method** was much faster than Gauss-Seidel in total runtime, even though it took more iterations to finish. This happened because Jacobi’s math can be fully vectorized in MATLAB, allowing it to calculate everything at once using fast, built-in array operations.
- **The Gauss-Seidel Method** performed poorly because it had to calculate every single point one by one using a slow loop (`for i = 1:N`). This stopped MATLAB from using its automatic speed optimizations. The extra time MATLAB wasted running this loop row-by-row completely wiped out the advantage of its better theoretical convergence rate.

6 Conclusion

For heat equation problems where the grid size and time steps do not change, LU Decomposition is the best choice because it calculates the main matrix math just once at the beginning. If the grid becomes so massive that the computer runs out of memory, you have to switch to iterative methods. In programs like MATLAB, the Jacobi method is actually better than Gauss-Seidel under these conditions because its math can be fully vectorized to run all at once, whereas Gauss-Seidel forces the computer to calculate values one by one.